

1. Day 4 – Your First Lightning Web Component (LWC)

Part 1 – Understanding the Basics

1. Why do users need a graphical interface?

A Graphical User Interface (GUI) helps users interact with Salesforce in a simple and intuitive way. Instead of using programming commands or database queries, users can complete tasks through buttons, forms, menus, and tables, making the application easier to use.

2. Why can't users directly execute SOQL queries?

Allowing users to run SOQL queries directly is not recommended because most users are unfamiliar with query syntax, incorrect queries can affect system performance, and direct database access may violate security policies. The application retrieves the required information and displays it through the interface instead.

3. Why is JavaScript required in LWC?

JavaScript provides the functionality of a Lightning Web Component. It handles user interactions, processes data, validates input, communicates with Apex classes, and updates the interface dynamically whenever actions are performed.

4. What are the responsibilities of the UI?

The User Interface is responsible for displaying information and enabling user interaction. It presents forms, buttons, tables, and messages, collects user input, performs basic validation, sends requests to the server, and displays appropriate responses.

5. What responsibilities should be handled by Apex?

Apex is responsible for server-side processing. It performs database operations, executes SOQL queries, applies business logic, processes records, manages transactions, enforces security, and supplies data to Lightning Web Components.

Part 2 – Components of LWC

Every Lightning Web Component is made up of three essential files.

1. studentHome.html

This file creates the visual layout of the component. It contains headings, labels, input fields, buttons, images, and the content displayed to users.

2. studentHome.js

This file manages the component's functionality. It processes user actions, stores values, handles events, interacts with Apex methods, and controls the overall behavior of the component.

3. studentHome.js-meta.xml

This configuration file specifies where the component can be used in Salesforce. It makes the component available in Lightning App Builder and defines supported targets such as Home Pages, App Pages, and Record Pages.

Quick Summary

File	Purpose
studentHome.html	Creates the user interface
studentHome.js	Implements the component logic
studentHome.js-meta.xml	Configures where the component is available

Part 3 – Hands-on Activity

Objective

Build your first Lightning Web Component for the Placement Management System and add it to a Lightning page.

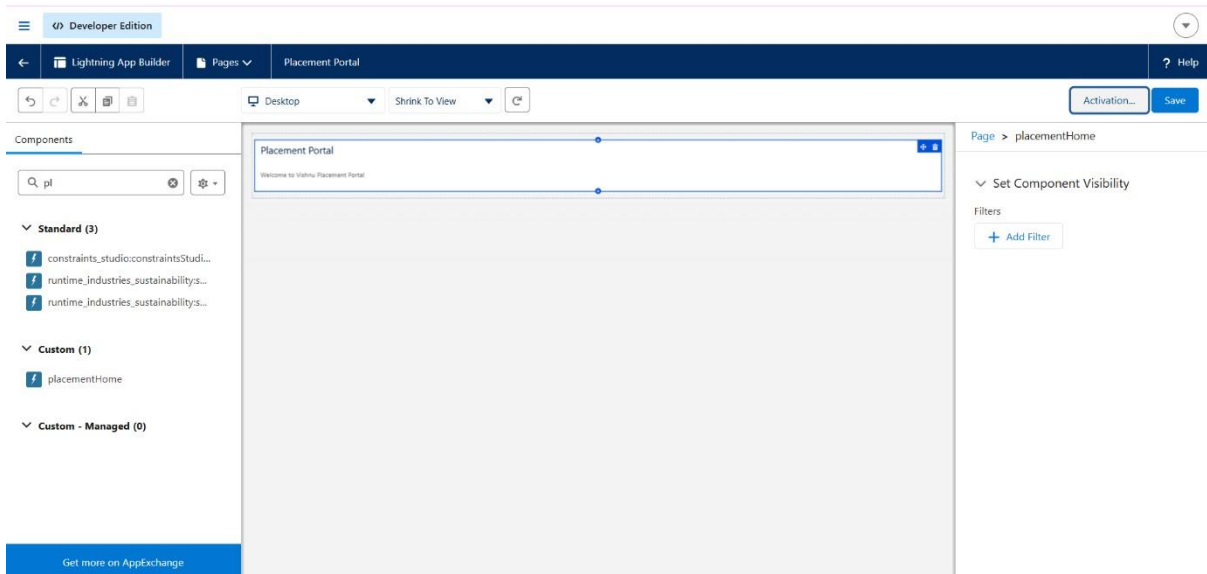
Component Name

placementHome

Expected Output

Placement Portal

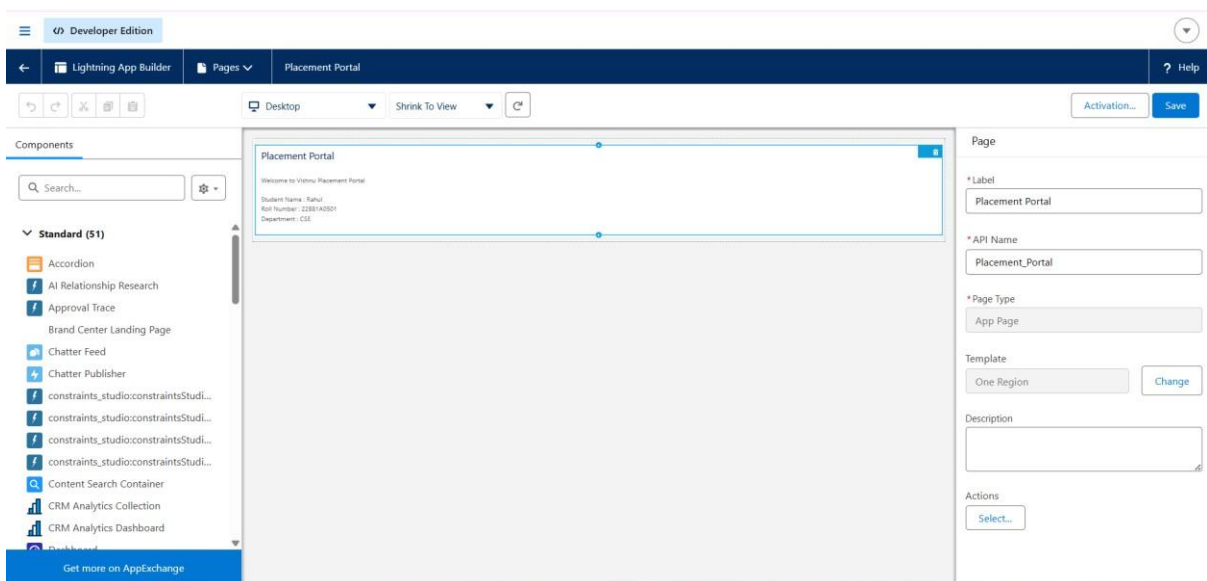
Welcome to the Student Placement Portal.



Part 4 – Hands-on Activity 2

Objective

To create Student Name, Roll Number, and Department variables in an LWC, display their values on the screen, and observe how the UI changes when the variable values are updated.



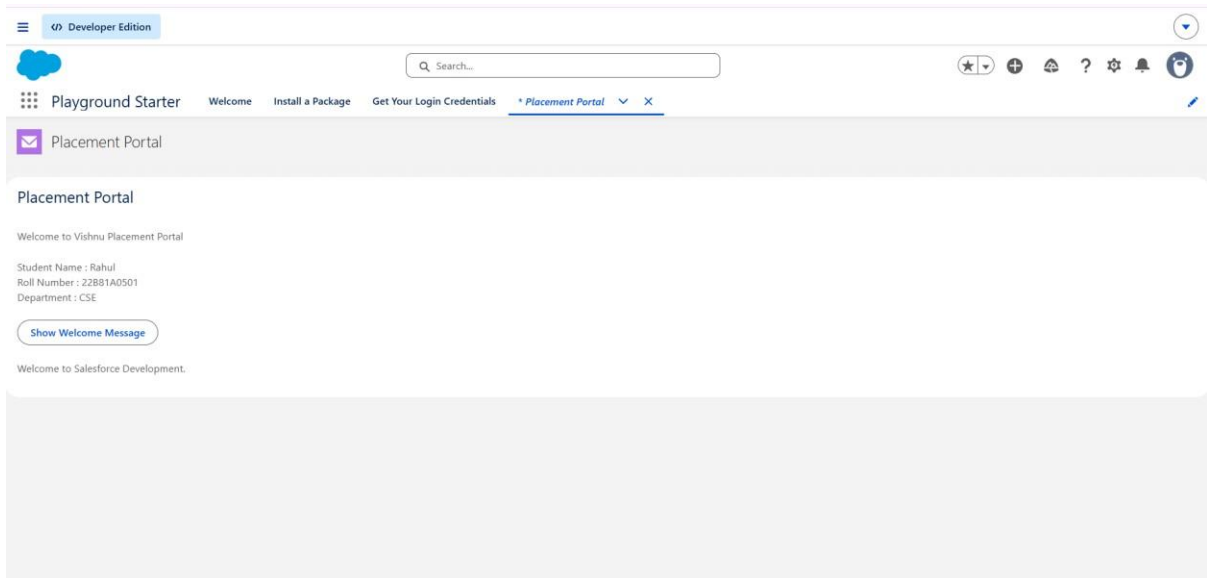
Part 5 – Hands-on Activity 3

Objective

To create a button in an LWC, handle a click event using JavaScript, and display a welcome message dynamically.

Initially, only the Show Welcome Message button appears.

After clicking the button, it displays: Welcome to Salesforce Development.



Part 6 – Hands-on Activity 4

Objective

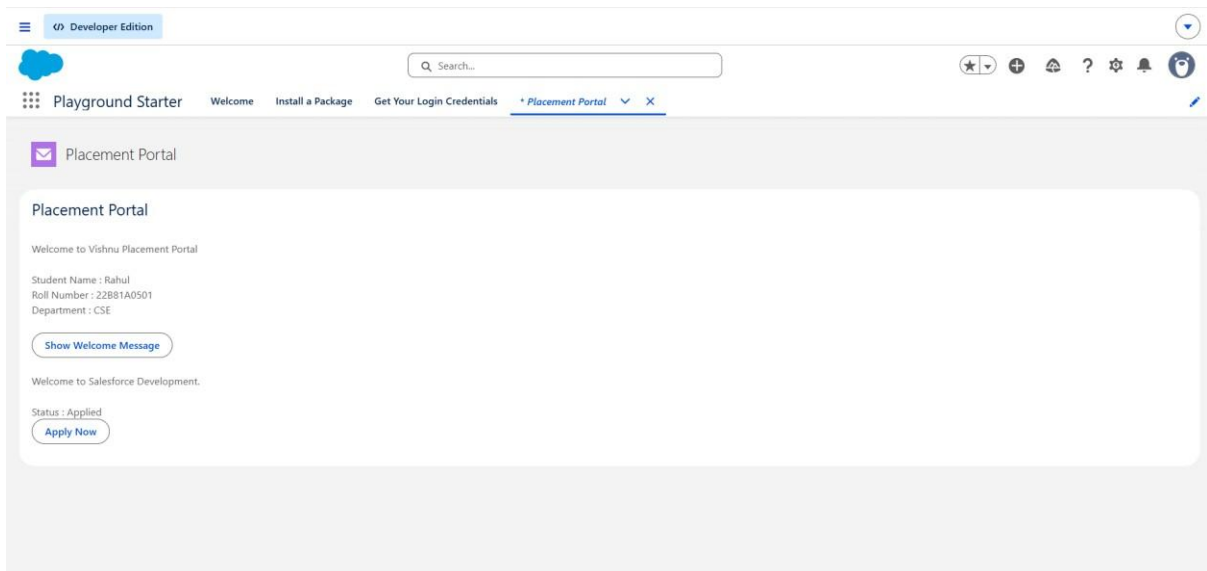
To understand how JavaScript can dynamically change data in an LWC without using Apex or a database.

Before clicking the button:

Status : **Not Applied**

After clicking Apply

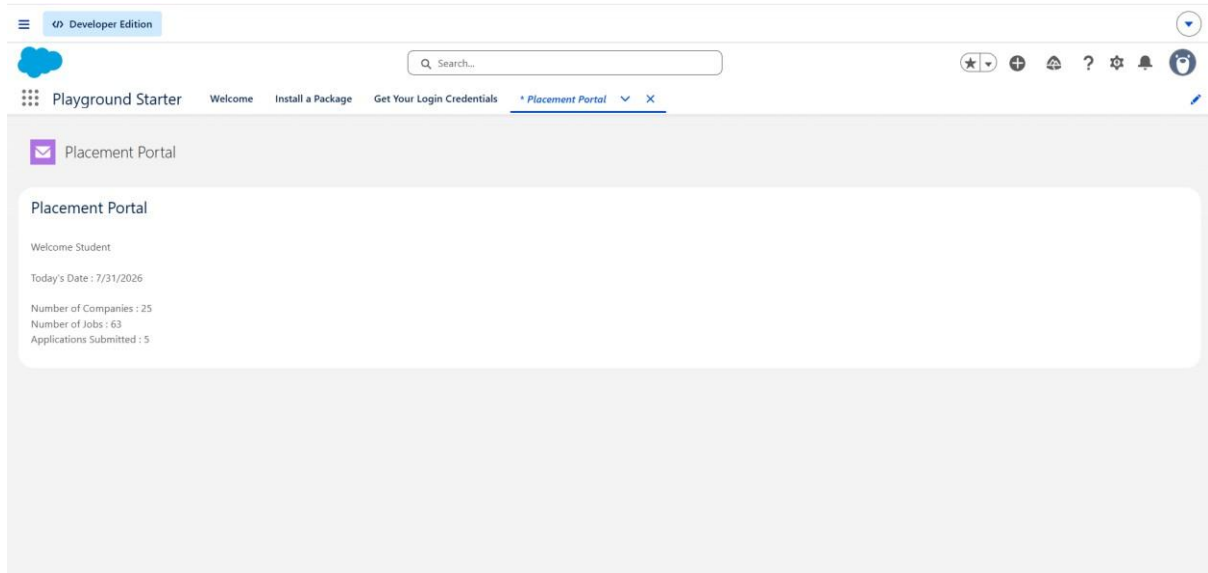
Status : **Applied**



Part 7 – Mini Project Enhancement

Objective

To create the first screen of the Placement Management System using LWC and display placement information using hard-coded JavaScript values.



Part 8 – Understanding Data Binding

Objective

To understand data binding in LWC by displaying a JavaScript variable in HTML and observing how the UI changes when the variable value is changed.

